

Adversarial Red-Teaming of LLM Evaluation Rubrics

NYU Tandon School of Engineering — ECE Master’s Thesis

Advisor: Prof. Chinmay Hegde — Expected May 2026

1 Introduction

Modern AI alignment pipelines rely heavily on automated LLM judges to evaluate model outputs at scale. These judges are typically paired with structured evaluation rubrics—sets of criteria that decompose a qualitative question (“is this a good response?”) into a set of assessable dimensions. Rubric-conditioned judges are increasingly used as a scalable substitute for human preference labeling in reinforcement learning from human feedback (RLHF) and related training regimes.

A fundamental challenge in this setting is that many real-world preference judgments are *non-verifiable*: there is no objective ground-truth metric to fall back on, only human preferences. A rubric is only as reliable as the criteria it specifies—a poorly specified rubric will lead a judge to produce internally inconsistent preference orderings, regardless of the judge model’s underlying capability. This makes rubric quality a critical bottleneck, and rubric weaknesses difficult to detect without a mechanism for exposing them systematically.

The central question driving this thesis is:

“Can we automatically identify weaknesses in a rubric by finding responses that expose inconsistencies in the preference ordering it induces, and can we use those exposures to iteratively improve the rubric?”

This thesis develops an adversarial framework to answer this question. The core insight is that **preference transitivity violations**—cases where a judge’s rubric-conditioned preferences are mutually contradictory across a triplet of responses—serve as a rigorous, self-contained signal of rubric weakness, requiring no external ground truth. By training a model to find such violations and a rubric generator to patch them, we construct an iterative loop that produces increasingly robust evaluation criteria, operating entirely within the non-verifiable preference domain.

2 Background

2.1 Rubric-Conditioned LLM Judges

Rubric-conditioned evaluation builds on the paradigm of LLM-as-a-judge, where a language model is prompted to compare or score responses according to a structured set of criteria. Unlike free-form evaluation prompts, rubrics provide explicit, decomposed dimensions (e.g., factual accuracy, clarity, task adherence) that are meant to anchor the judge’s scoring behavior and reduce variance.

A key development motivating this work is the **Rubric-ARM** framework [1], which introduces a model family consisting of a rubric generator and a rubric-conditioned judge, jointly optimized via alternating reinforcement learning from preference feedback. Critically, rather than using a fixed universal rubric, Rubric-ARM abstracts rubric specification into a *learned generative model*: given any user instruction, the rubric generator produces a task-specific rubric tailored to that prompt. This design is a significant departure from prior work—it means rubric quality can scale with the diversity of the instruction distribution, without requiring manual rubric engineering. This makes rubric-conditioned evaluation substantially more practical in diverse, open-ended, and non-verifiable preference domains where a single fixed rubric would be inadequate. Rubric-ARM achieves state-of-the-art performance among white-box methods across a range of alignment benchmarks including RewardBench (Chat/Chat-Hard), RM-Bench, and a suite of instruction-following benchmarks (PPE-IFEval, FollowBench, IFBench, RewardBench2), and significantly improves downstream policy alignment in both offline and online RL settings.

2.2 Preference Transitivity as a Robustness Criterion

Preference transitivity is a fundamental consistency requirement for any rational evaluation system: if a judge prefers response A to response B , and B to C , it should prefer A to C . Violations of this property indicate internal inconsistency in the judge’s evaluation criteria and represent a concrete, self-certifying failure mode—one that requires no external ground truth to detect.

This work operationalizes transitivity violations as the exploit signal. Given a preference dataset with labeled preferred (chosen, A) and dispreferred (rejected, B) response pairs, we define a **transitivity violation** as the existence of an exploit response C such that:

$$\text{judge}(C \succ A) = \text{True} \quad \text{and} \quad \text{judge}(C \prec B) = \text{True}$$

That is, the exploit response is simultaneously ranked *above* the human-preferred response and *below* the human-dispreferred response under the same rubric and judge. This is a direct contradiction in the judge’s own preference ordering and a clear indicator of rubric misspecification, requiring no external quality oracle to verify.

2.3 GRPO for Adversarial Training

Group Relative Policy Optimization (GRPO) is an RL algorithm developed for training LLMs with verifiable reward signals. It removes the need for a separate value/critic network by computing advantages from group-level returns: for each input, G responses are sampled and

ranked relative to each other, producing a reward-normalized policy gradient signal. GRPO is particularly well-suited to the sparse, preference-based reward settings used in this work.

Both the exploit model and the rubric generator are trained with GRPO, using TRL’s GRPOTrainer.

3 Methodology

3.1 System Overview

The framework implements a two-phase iterative loop over a preference dataset of (instruction, chosen response A , rejected response B) triples. The loop alternates between an **exploit phase** (Phase A) that probes the current rubric for transitivity weaknesses, and a **patch phase** (Phase B) that updates the rubric generator to correct those weaknesses.

A pretrained RubricARM-8B rubric generator provides the initial rubric for each example. In Phase A, rubrics are generated from the *current* rubric generator checkpoint and cached per example—this isolates the exploit training signal from variance introduced by rubric regeneration and ensures a stable adversarial target. In Phase B, the rubric generator is updated to produce rubrics that survive adversarial pressure, evaluated over both the original preference pairs and the newly discovered exploit responses.

3.2 Phase A: Exploit Training

The exploit model is trained via GRPO to generate responses that induce preference transitivity violations in the rubric-conditioned judge. The training setup is as follows:

- **Input per training step:** An (instruction, chosen A , rejected B , rubric) quadruple drawn from the cached dataset.
- **Rollout:** $G = 16$ candidate responses (Response C) are sampled from the exploit model for each training example.
- **Reward function:** Binary verifiable reward—reward = 1.0 if the judge scores $C \succ A$ and $C \prec B$ (transitivity violation); reward = 0.0 otherwise.
- **Judge:** Llama-3.3-70B-Instruct, conditioned on the cached rubric. Each reward computation requires two pairwise comparisons per candidate: $\text{Pair}(A, C)$ and $\text{Pair}(B, C)$, as illustrated in Figure 1.

3.3 Phase B: Rubric Patch Training

Once Phase A converges—the exploit model reliably produces transitivity-violating responses—the rubric generator is fine-tuned via GRPO to patch the identified weaknesses. The setup is as follows:

- **Policy:** The rubric generator (initialized from RubricARM-8B-Rubric) is the GRPO policy; its actions are generated rubrics.
- **Environment:** The judge model evaluates responses under the proposed rubric.

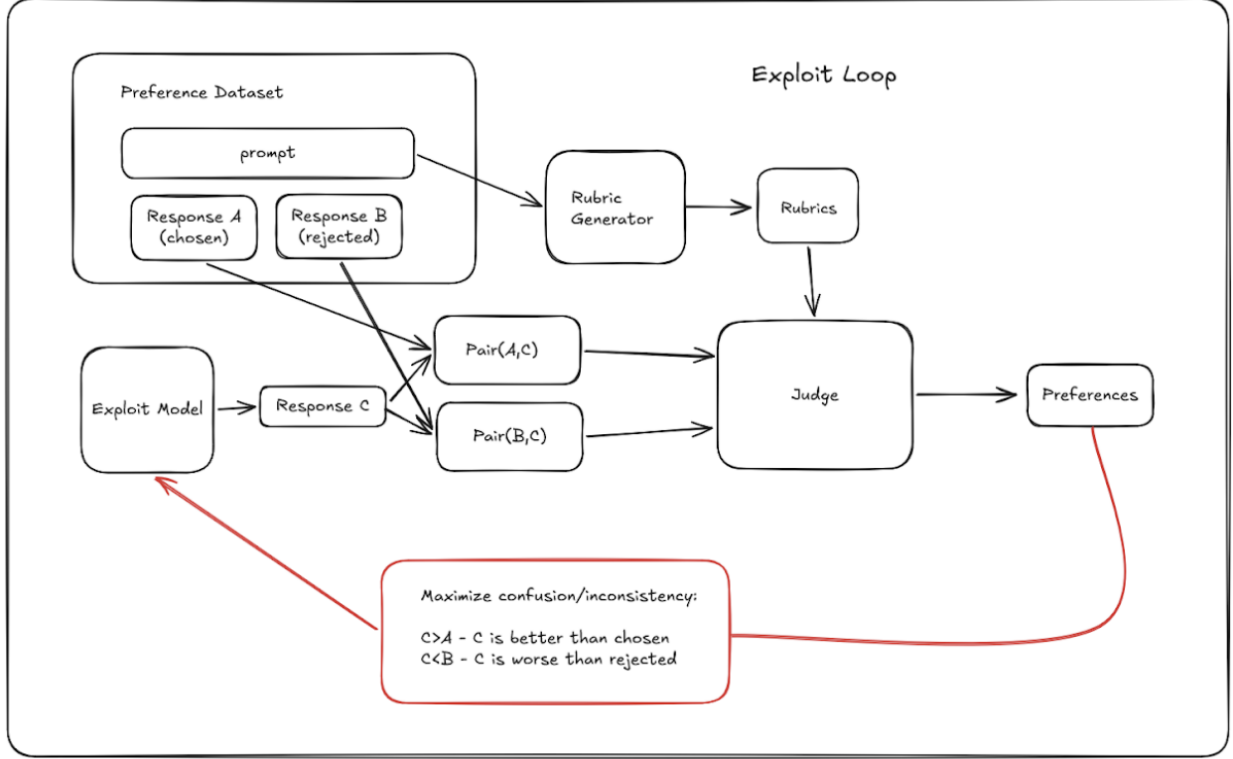


Figure 1: Exploit loop (Phase A). The exploit model generates Response C for each prompt, which is paired with the chosen (A) and rejected (B) responses and evaluated by the rubric-conditioned judge. The reward signal drives the exploit model toward responses that simultaneously satisfy $C \succ A$ and $C \prec B$ under the judge’s preference ordering.

- **Reward:** The rubric’s reward is computed over two evaluation conditions, as shown in Figure 2:
 - *Dataset preference preservation:* The judge should correctly identify $A \succ B$ on the original preference pair.
 - *Triplet consistency:* Given the exploit response C , the judge’s preferences over (A, B, C) should be internally consistent—any of $A \succ B \succ C$, $C \succ A \succ B$, or $A \succ C \succ B$ is acceptable; the transitivity violation $C \succ A$ and $C \prec B$ is not.
- **Replay buffer:** The reward dataset combines (i) original preference pairs, (ii) exploit responses from all prior Phase A iterations, and (iii) newly generated exploit responses from the current Phase A checkpoint. This prevents catastrophic forgetting—without historical replay, the patched rubric may correct current exploits while becoming re-vulnerable to prior ones.
- **Convergence:** The loop terminates when the exploit model cannot produce transitivity-violating responses under the updated rubric within a fixed number of rollout attempts.

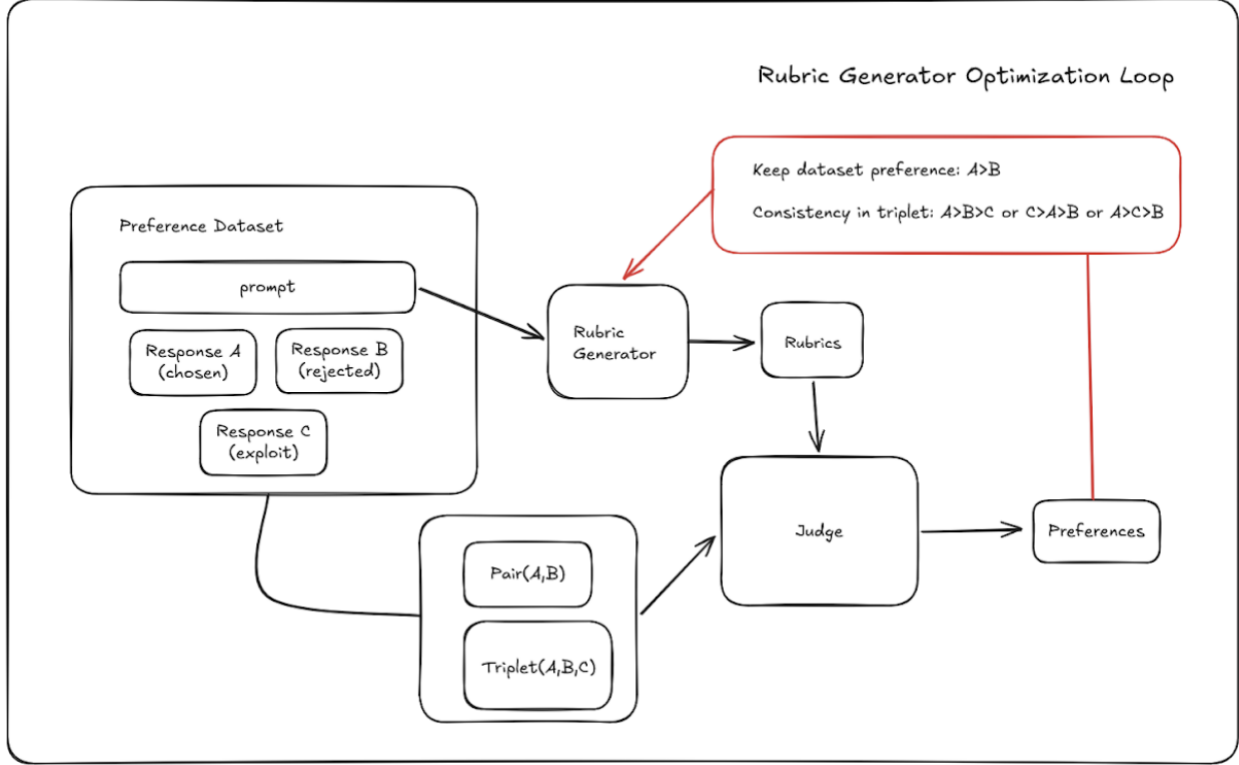


Figure 2: Rubric generator optimization loop (Phase B). The rubric generator is updated to produce rubrics that, when used by the judge, restore consistency across both the original preference pair (A, B) and the exploit triplet (A, B, C) . The reward requires both dataset preference preservation $(A \succ B)$ and transitivity consistency in the triplet.

The Phase B design is directly inspired by the Rubric-ARM framework [1], which treats rubric generation as a policy optimization problem. This work extends that approach by closing the adversarial loop—rather than optimizing rubrics against a fixed dataset of known hard cases, the rubric is iteratively hardened against an actively adapting exploit model, operating entirely without external ground-truth labels.

References

- [1] Ran Xu, Tianci Liu, Zihan Dong, Tony Yu, Ilgee Hong, Carl Yang, Linjun Zhang, Tao Zhao, and Haoyu Wang. *Alternating Reinforcement Learning for Rubric-Based Reward Modeling in Non-Verifiable LLM Post-Training*. arXiv:2602.01511, 2026.